赛博画师

1．模型预训练与微调

1.1 **模型预训练**与**模型微调**相关知识

预训练：

先让网络在超大规模的数据集（如著名的 ImageNet，拥有 120 万张日常图片、1000 种物体）上训练几周，网络在这个过程中学会了“看图的基本能力”——比如识别线条、斑点、阴影和常见物体的轮廓
用比较专业的语言来说就是模型的卷积核等参数对图片处理后每一层得到的特征图张量能够与图像的某些特征（色彩，纹理，线条，轮廓等等）具有良好的映射关系

微调：

拿上述已经训练好的模型做起点，用我们几千张特定数据继续训练（比如花卉数据集）

只训练某几层，让它适应新任务

直观上的理解：

cnn网络是由一层一层堆起来的，每一层感受野由小变大，更深层看到的也就越多

浅层网络的感受野小，看到的像素少，能够捕捉较为局部和具体的特征（比如图片的等等这些边缘、颜色、角点与**像素强相关的特征**）（**这些特征与像素紧密相连**），但是对于较为宏观和全局的特征难以捕捉（超出视野了做不到捏）

深层网络的感受野大，看到的像素多，能够捕捉较为宏观和抽象的特征（比如图片的整体内容，物品的抽象类别等等这些与**像素弱相关的特征**）（**这些特征越来越脱离像素本身，变得更概念化、语义化**），但是对于较为局部和具体的特征就难以捕捉（被宏观的图像结构主导了）

这也就是cnn网络提取特征的分工，而之前的训练就是要模型从零开始学会提取特征，从而分类

而**预训练**就是在超大规模的数据集上已经学会了如何去提取特征，我们风格迁移任务需要这样优秀的特征提取器，用于提取原图和风格图的特征

而**微调**常见做法是固定较浅层的参数（**浅层**学的是边缘、颜色、角点、纹理—这些是**几乎所有视觉任务通用**的，换任务也照样有用），调整较深层的参数（**深层**学的是物体、语义、类别相关的抽象概念—这些**和具体任务强相关**，换任务往往需要改）

微调的直觉：**浅层别动（通用知识保留），深层动一动（适配新任务）**

这样做好处在哪?

1. 正如前文所说，预训练网络已经学会了鲁棒的低级视觉特征，调整深层直击具体任务那一类图像的要害，效果好
2. 因为只用训练深层的数据，参数量小，跑得快，时间成本低
3. 参数量小，所需的图片量也就少

1.2 冻结参数

冻结参数是什么：

只改变某几层的参数，其他层的不改变或者参数都不改，直观上就像**参数被冻结了**（`p.requires_grad = False`）

常见的做法是冻结浅层的参数，微调深层的参数

也可以参数都不改，只当特征提取器（本题就是这么做的，因为微调就要很多的风格图来训练，而这个题只要一张内容图和一张风格图，完全不需要）

冻结参数有什么用：

1. **保留通用知识**：

   正如前文所说，浅层参数提取的是具象的特征，浅层学到的边缘、颜色、纹理是几乎所有视觉任务通用的，冻结它们就是保住这些"底层视觉能力"不被新数据破坏，如果不冻结，浅层一旦被小数据带偏，整个网络的特征提取基础就崩了

2. **防止过拟合**：

训练参数越多越容易过拟合，冻结浅层等于大幅削减可训练的参数量，模型被迫只学"新任务直接相关"的部分，对于噪声的拟合能力就变差了（相当于防止过拟合）

3. **省算力、省显存、省时间**：

   冻结后反向传播不用算和保存浅层的梯度，训练快很多，算力省的多，显存占用少（也就更省钱）

4. **提供灵活的模型容量选择**：

   冻结是我们根据具体任务可以自由选择的，也就是根据任务的具体要求决定模型有多少自由度去适配新任务，冻结越多，模型越"保守"；解冻越多，越"灵活"，但也越容易过拟合

   灵活的冻结选择带来了不同的调整尺度

模型微调时超参数该如何设置：

核心思想：参数变化小，抑制过拟合

1. **学习率**：

   比从头训练**小 1~2 个数量级**（如 1e-4 ~ 1e-5）

   原因：深层在超多的数据集下训练的已经很好了，预训练权重已经接近一个好解，学习者只需小幅调整，大步长会破坏这些特征并导致震荡

2. **Batch size**：

   尽量大

   原因：一个batch的数据多，取平均就能缓冲极端值带来的大梯度，也就是减小参数变化量

3. **轮数**：

   3~10左右

   原因：数据集小，轮数多了容易过拟合，轮数小避免在小数据集上强行过拟合

4. **权重衰减**：

   优化器选择好用的AdamW，权重衰减可适当加大，抑制过拟合

1.3 风格迁移领域预训练模型的作用：

预训练的模型带来两样东西：

1. **现成的特征提取器**（不用从零开始训练了）
2. **层次化的特征提取器**（**浅层给风格、深层给内容** —— 把两者分开，为后面的损失函数做铺垫）

常见模型有VGG和ResNet

VGG的结构（这里以VGG19为例）（输入图片以768大小为例）：

核心思想：**只用 3×3 小卷积核，靠"堆层"来提升表达能力**

VGG19分两部分，一个特征提取层（**feature**）（这个题要的），一个全连接层（**classifier**）

特征提取层：

特征提取层有37层，分为5个block（具体层太大就不展示了），每个block由卷积，池化，relu交替组成

block	索引	卷积数	输出通道	输出尺寸
block1	0~4	2	64	768×768
block2	5~9	2	128	384×384
block3	10~18	**4**	256	192×192
block4	19~27	**4**	512	96×96
block5	28~36	**4**	512	48×48

Table 1

每进一个 block → 通道数翻倍（或保持 512）→ 特征种类变多
每进一个 block → 尺寸减半（因为末尾有 pool）→ 空间信息被压缩

层索引命名规则：

xxxa_b:第a个block的第b个xxx操作层

例：**relu4_2**就是第四个block的第二个relu操作层，索引为22

ResNet与VGG的不同：

1. 残差跳连的新办法
2. 参数量少

总参数	卷积主体	全连接头
VGG19	143,667,240	20,024,384 (**13.9%**)
ResNet18	11,689,512	11,176,512 (95.6%)

Table 2

VGG19 的 1.43 亿参数里，1.24 亿（86%）都在全连接层

ResNet18 用全局平均池化替代了那个巨型全连接层，所以全连接头只有 51 万

使得算的更快。效率更高

2. 初识风格迁移

以下内容不是回答，只是思考（没有对问题正面回答，只是对于风格迁移的初步认识）：

2.1 风格迁移与图片分类都是图像处理，这两有什么不同呢?

	图片（花卉）分类	风格迁移
训练对象	**网络参数**（5800 万个）	**图片像素**（网络一个参数都不改）
需要多少数据	3457 张	**2 张**（一张提供风格，一张提供内容）
训练时长	30 轮约 1 小时	**几分钟**
产物	一个模型文件	**一张图片**
输出的维度	5 个数字（类别分数）	$3 \times H \times W$ 个数字（像素）

Table 3

正如上面所说，我们图片分类要的是改网络去适应图片，而风格迁移网络是我们**提取特征的工具**，即使改也只是小规模的改（微调），**真正改的参数是我们要生成的图片的像素**

而这里的损失函数就变成了衡量生成的图片与我们理想的图片的差距的函数

而我们理想的图片是什么？一张能完美包含A图内容和B图风格的图

损失函数也就是来衡量生成的图片与A图（内容图）内容差距和B图（风格图）

也就是对于**风格迁移**这样的新任务

我们找到了**要调整的参数**和**损失函数**

那还说啥了，直接启动!

2.2 六百六十六，还有第二关?

2.2.1 为什么抽象的风格和内容能被提取?

引用前文：

浅层网络的感受野小，看到的像素少，能够捕捉较为局部和具体的特征（比如图片的等等这些边缘、颜色、角点与**像素强相关的特征**）（**这些特征与像素紧密相连**），但是对于较为宏观和全局的特征难以捕捉（超出视野了做不到捏）

深层网络的感受野大，看到的像素多，能够捕捉较为宏观和抽象的特征（比如图片的整体内容，物品的抽象类别等等这些与**像素弱相关的特征**）（**这些特征越来越脱离像素本身，变得更概念化、语义化**），但是对于较为局部和具体的特征就难以捕捉（被宏观的图像结构主导了）

什么是风格，什么是内容?

同样内容的一张图风格不一样会发生什么？（Gemini生成的，凑活看吧）

Figure 1

Figure 2

远远来看这两张图描述了同一个事情，但是感受上风格不同

放大，你就会发现第一张颜色过渡更平滑，第二张添加了更多细节，在某些部分像素的变化会更剧烈

整体感受上画面的明暗，取色也不同

也就是在微小的线条，纹理上，中等的不同物体过渡和细节，宏大的整体明暗与取色上均有所不同，**"风格"是多个尺度上的东西**，不同层提取到了不同层次的风格

也就是我们要同时考虑**由浅到比较深**的层的特征图，在VGG19中用relu1_1，relu2_1，relu3_1，relu4_1，relu5_1每个block的层**一起来评估风格**

只用浅层	只用深层	5 层全用
只有细碎纹理，缺大色块	只有粗糙色块，缺笔触细节	**笔触和色块都有**

Table 4

而内容是对图像整体的把握（发生了什么，有什么东西等等），一定程度上超越了像素，是一种抽象语义，这需要更深的层数才能提取到（也不能太深）

候选	问题
太浅（relu1_1、relu2_1）	**只有纹理、没有语义**
太深（relu5_1）	特征图只有 48×48，**空间分辨率太低** — 位置约束太弱，形状会飘
relu4_2（96×96，512 通道）	**既有语义（知道"是花"）、又有足够空间分辨率（知道"在哪"）**

Table 5

relu4_2刚刚好

这样抽象的风格和内容就可以用不同层提取的特征来表述

2.2.2 英雄难过数学关~

正如前面任务的损失函数有个具体公式一样，我们风格迁移也要有公式：

$$L_{total} = \alpha L_{content} + \beta L_{style} + \gamma L_{tv}$$

可以看到，这个公式由三部分组成：

内容损失（Content Loss），风格损失（Style Loss），总变分损失（Total Variation Loss）

总损失就是三者的加权和

内容损失（Content Loss）：

在本题中我们仅用relu4_2的结果来衡量内容

度量：生成图的"形状布局"和内容图差多少

$$L_{content} = \frac{1}{2} \sum_{i,j} (F_{ij} - P_{ij})^2$$

说人话：生成图与内容图经过relu4_2后特征张量的差张量，差张量的元素平方和

风格损失（Style Loss）：

在本题中我们用relu1_1，relu2_1，relu3_1，relu4_1，relu5_1一起来衡量风格

度量：生成图的风格和风格图差多少

关键区别：它不比较特征图本身，而是比较 Gram 矩阵：

$$L_{style} = \sum_{l} w_l \cdot L^l_{style} \qquad L^l_{style} = \frac{1}{4N_l^2 M_l^2} \sum_{i,j} \left(G^l_{ij} - A^l_{ij}\right)^2$$

说人话：生成图和风格图经过relux_1后 Gram 矩阵差的元素平方和，再加到一起

总变分损失（Total Variation Loss）：

$$L_{tv} = \sum_{i,j} \left(|x_{i,j+1} - x_{i,j}| + |x_{i+1,j} - x_{i,j}| \right)$$

度量：生成图"相邻像素差多少"

三个损失的分工总结：

损失	比较对象	取自	关心位置吗	作用
内容	特征图**本身**	conv4_2（深层）	**关心**	保住"画的是什么"
风格	**Gram 矩阵**	conv1_1~5_1（多层）	**完全不关心**	搬来"画风"
TV	相邻**像素**	生成图自身	关心（局部）	让图平滑

Table 6

有了上面的损失函数，利用优化器调整**生成图的像素**来减少损失函数，从而接近我们理想中图片的样子，最终模型输出一张理论上融合内容图内容和风格图风格的图片（**此时还是张量，需要通过PIL等库来转换和保存**）

3．代码细节

下面是经过debug多次的代码（有点长）

用了LBFGS这一新的优化器，没用mask（用了mask的在附加回答里面）

```python
"""赛博画师风格迁移python文件
用预训练 VGG19 当特征提取器
优化器用 LBFGS（二阶 + 线搜索）
没用mask
原图放pictures文件夹下，风格图和内容图放同一set的style_image和content_image下
本文件实现对内的图扫描并生成风格迁移图，放在对应output/set下
目录约定（自动扫描，加新 set 只建文件夹放图，不用改代码）：
    pictures/set1/content_image/内容图
    pictures/set1/style_image/风格图
    output/set1/result.png              最终生成图
    output/set1/loss.csv                每步损失
    output/set1/compare/iter_*.png      过程图（对数间隔，起点的 iter_0000 是原图）
"""
# 库的引入
import csv
import os
import time
import torch
import torch.nn.functional as F
import torchvision.models as models
import torchvision.transforms as T
from PIL import Image

# 路径
SRC = os.path.dirname(os.path.abspath(__file__))
PROJECT = os.path.dirname(SRC)
PICTURES = os.path.join(PROJECT, "pictures")
OUTPUT = os.path.join(PROJECT, "output")

# 超参数
IMG_SIZE = 768          # 生成图边长
NUM_ITERS = 100         # LBFGS 外层步数（每步内部还会做多次子迭代）
NUM_SNAPSHOTS = 8       # 过程图张数（对数间隔，0 = 不存）
LBFGS_LR = 1.0          # 线搜索的初始尺度
LBFGS_MAX_ITER = 20     # 每次 step 内部最多几次子迭代
LBFGS_HISTORY = 10      # 用最近多少步梯度近似曲率

# 三个损失的权重
CONTENT_WEIGHT = 1.0    # alpha（大了风格弱白训练，小了原图结构丢失）
STYLE_WEIGHT = 1e4      # beta（大了风格吃掉原图，严重扭曲，小了风格弱了）
TV_WEIGHT = 1e-4        # gamma（控制平滑程度，大了变糊，小了噪点多）

```

```python
# 层索引
CONTENT_LAYERS = [22]                       # relu4_2: 取内容
STYLE_LAYERS = [1, 6, 11, 20, 29]           # relu1_1~relu5_1: 取风格

# 归一化参数
IMAGENET_MEAN = [0.485, 0.456, 0.406]
IMAGENET_STD = [0.229, 0.224, 0.225]

IMAGE_EXTS = (".jpg", ".jpeg", ".png", ".bmp", ".webp", ".avif")

# part1:归一化及其逆变换
def to_norm(tensor):
    """对张量归一化"""
    mean = torch.tensor(IMAGENET_MEAN,device=tensor.device).view(1,3,1,1)
    std = torch.tensor(IMAGENET_STD,device=tensor.device).view(1,3,1,1)
    return (tensor - mean) / std

def to_tensor(tensor_normed):
    """逆归一化"""
    mean = torch.tensor(IMAGENET_MEAN,device=tensor_normed.device).view(1,3,1,1)
    std = torch.tensor(IMAGENET_STD,device=tensor_normed.device).view(1,3,1,1)
    return ((tensor_normed * std) + mean).clamp(0,1)

def to_PIL_image(tensor):
    """tensor变PIL图片（先 clamp 到 [0,1] —— 优化过程中数值会跑出合法范围，
    不 clamp 的话界值转 uint8 时会被截断成孤立极值像素，就是彩色噪点）"""
    return T.ToPILImage()(tensor.detach().clamp(0,1).squeeze(0).cpu())

# part2:读图和扫描
def load_image(path,device,size=IMG_SIZE):
    """读一张图,返回归一化张量 (1, 3, size, size)"""
    I = Image.open(path).convert("RGB")
    tf = T.Compose([T.Resize(size),T.CenterCrop(size),T.ToTensor()])
    return to_norm(tf(I).unsqueeze(0).to(device))

def discover_sets(pictures_path=PICTURES):
    """把pictures的set打包,返回全是元组的list,(set,内容图path,风格图path)"""
    found = []
    if not os.path.isdir(pictures_path):
        return found
    for name in sorted(os.listdir(pictures_path)):
          set_dir = os.path.join(pictures_path, name)
          paths = []
          for dir_name in ["content_image", "style_image"]:
              folder = os.path.join(set_dir,dir_name)
              pictures = sorted(p for p in os.listdir(folder) if p.lower().endswith(IMAGE_EXTS))
              paths.append(os.path.join(folder,pictures[0]))
          found.append((name,paths[0],paths[1]))
    return found

# part3:VGG19特征提取
def load_vgg(device):
    """加载预训练VGG19的卷积部分,先冻结所有参数"""
    vgg = models.vgg19(weights=models.VGG19_Weights.IMAGENET1K_V1).features
    return vgg.to(device).eval().requires_grad_(False)


def get_features(vgg,x,track=False):
    """让x穿过vgg特征提取部分,返回{层索引:特征图张量}"""
    feats = {}
    with torch.set_grad_enabled(track):
        for i,layer in enumerate(vgg):
            x = layer(x)
            if i in CONTENT_LAYERS or i in STYLE_LAYERS:
                feats[i] = x
```

```
            if i >= max(CONTENT_LAYERS + STYLE_LAYERS):
                break
    return feats

# part4:数学公式！！！(骇死我了)
def gram_matrix(tensor):
    """Gram 矩阵"""
    b,c,h,w = tensor.shape
    tensor = tensor.view(b,c,h * w)
    return torch.bmm(tensor,tensor.transpose(1,2)) / (b * c * h * w)

def loss_calculate(feats,content_target, style_target, gen_normed):
    """计算loss"""
    c = sum(F.mse_loss(feats[i], content_target[i]) for i in CONTENT_LAYERS)
    s = sum(F.mse_loss(gram_matrix(feats[i]) , gram_matrix(style_target[i])) for i in STYLE_LAYERS)
    t = (torch.mean(torch.abs((gen_normed[:,:,:-1,:] - gen_normed[:,:,1:,:])))) +
         torch.mean(torch.abs((gen_normed[:,:,:,:-1] - gen_normed[:,:,:,1:]))))
    return c, s, t,  CONTENT_WEIGHT * c + STYLE_WEIGHT * s + TV_WEIGHT * t

# part5:存储风格变迁图和csv
def get_snapshot_steps(iters=NUM_ITERS,count=NUM_SNAPSHOTS):
    """按对数间隔挑出存过程图的步数（前期密、后期疏）"""
    if count <= 0:
      return set()
    steps = {max(1,round(iters ** (i / count))) for i in range(1,count + 1)}
    return steps | {iters}

def save_loss_csv(record,path):
    with open(path,"w",encoding="utf_8",newline="") as f:
        w = csv.writer(f)
        w.writerow(["iter","content_loss","style_loss","tv_loss",
                    "weighted_content", "weighted_style",
                    "weighted_tv", "total"])
        for step,c,s,t,total in record:
            w.writerow([step,c,s,t,c * CONTENT_WEIGHT,s * STYLE_WEIGHT,t * TV_WEIGHT,total])
    print(f"已把对应set的csv存入{path}")

# part6:核心流程
def transfer(vgg,content,style,snapshot_dir=None,snapshot_steps=None,
             patience=5,min_delta=1e-4):
    """干什么的：以内容图为起点，用 LBFGS 优化 NUM_ITERS 步像素
    输入 vgg,内容图张量，风格图张量，风格变迁文件夹路径，风格变迁要存储的步数
    返回 生成图张量， 最优总损失， 最优步数， 损失记录， 实际步数
    """
    gen = to_tensor(content).clone().requires_grad_(True)
    optimizer = torch.optim.LBFGS([gen], lr=LBFGS_LR, max_iter=LBFGS_MAX_ITER,
                                  history_size=LBFGS_HISTORY,
                                  line_search_fn="strong_wolfe")

    content_target = get_features(vgg, content)
    style_target = get_features(vgg, style)

    def closure():
        with torch.no_grad():
            gen.clamp_(0, 1)
        optimizer.zero_grad()
        x = to_norm(gen)
        feats = get_features(vgg,x,track=True)
        c, s, t, total = loss_calculate(feats,content_target,style_target,x)
        loss = total
        loss.backward()
        closure.losses = (c.item(), s.item(), t.item(),total.item())
        return loss
    with torch.no_grad():
        x0 = to_norm(gen)
```

```
            c0,s0,t0,total0 = loss_calculate(get_features(vgg,x0),content_target,style_target,x0)
        record = [(0,c0.item(),s0.item(),t0.item(),total0.item())]
        best_gen, best_total, best_step = None, float("inf"), 0
        prev_total, stale, done = 1e6, 0, 0

        for step in range(1,NUM_ITERS +1):
            optimizer.step(closure)
            with torch.no_grad():
                gen.clamp_(0, 1)
            c,s,t,total = closure.losses
            record.append((step, c, s, t, total))
            done = step

            if total < best_total:
                best_total, best_step = total, step
                best_gen = gen.detach().clone()
            improve = (prev_total - total) / (abs(prev_total)+1e-8)
            stale = stale + 1 if improve < min_delta else 0
            prev_total = total

            print(f"第{step:6d}/{NUM_ITERS}步 内容损失 {c:8.4f} 风格损失 {s:.6f} tv损失 {t:.6f} 总损失
{total:.9f}")

            if snapshot_dir and step in snapshot_steps:
                to_PIL_image(gen).save(os.path.join(snapshot_dir,f"iter_{step:04d}.png"))
            if stale >= patience:
                print(F"早停啦，当前{step}轮")
                break
        print(f"finished 实际跑了{done}轮，最优在第 {best_step} 步，总损失 {best_total:.4f}")
        return best_gen,best_step,best_total,record,done

    # 主流程
    def main():
        device = torch.device("cuda" if torch.cuda.is_available() else "cpu")
        print(f"训练设备 {device} 尺寸 {IMG_SIZE}")
        print("训练启动！！！")

        sets = discover_sets()
        snapshot_steps = get_snapshot_steps()
        vgg = load_vgg(device)
        print(f"此次共有{len(sets)}个set")

        for set_name,content_path,style_path in sets:
            print(f"对{set_name}进行生图")
            t0 = time.time()
            content = load_image(content_path,device)
            style = load_image(style_path,device)
            output_set_path = os.path.join(OUTPUT,set_name)
            output_set_snapshot_dir = os.path.join(output_set_path,"compare")
            if snapshot_steps:
                os.makedirs(output_set_snapshot_dir,exist_ok=True)

     to_PIL_image(to_tensor(content)).save(os.path.join(output_set_snapshot_dir,"iter_0000_content.png"
    ))

            gen, best_step, step, record, done = transfer(vgg, content, style, output_set_snapshot_dir,
    snapshot_steps)
            os.makedirs(output_set_path,exist_ok=True)
            to_PIL_image(gen).save(os.path.join(output_set_path, "result.png"))
            save_loss_csv(record, os.path.join(output_set_path, "loss.csv"))
            print(f"  {set_name}用时{time.time() - t0:.2f}秒")

        print("全部完成")

    if __name__ == "__main__":
```

```
234      main()
235
```

Fence 1

说明：

实现对于多组内容图和风格图输出生成图：

set1：contnet1，style1

set2：contnet2，style2

……

本题示例选取了三组：

set1—梵高星空油画
set2—二次元动画风格
set3—夕阳风格
每组采用同一张内容图，方便对比风格

下面是图片展示，问题回答在后面：

共同的内容图：

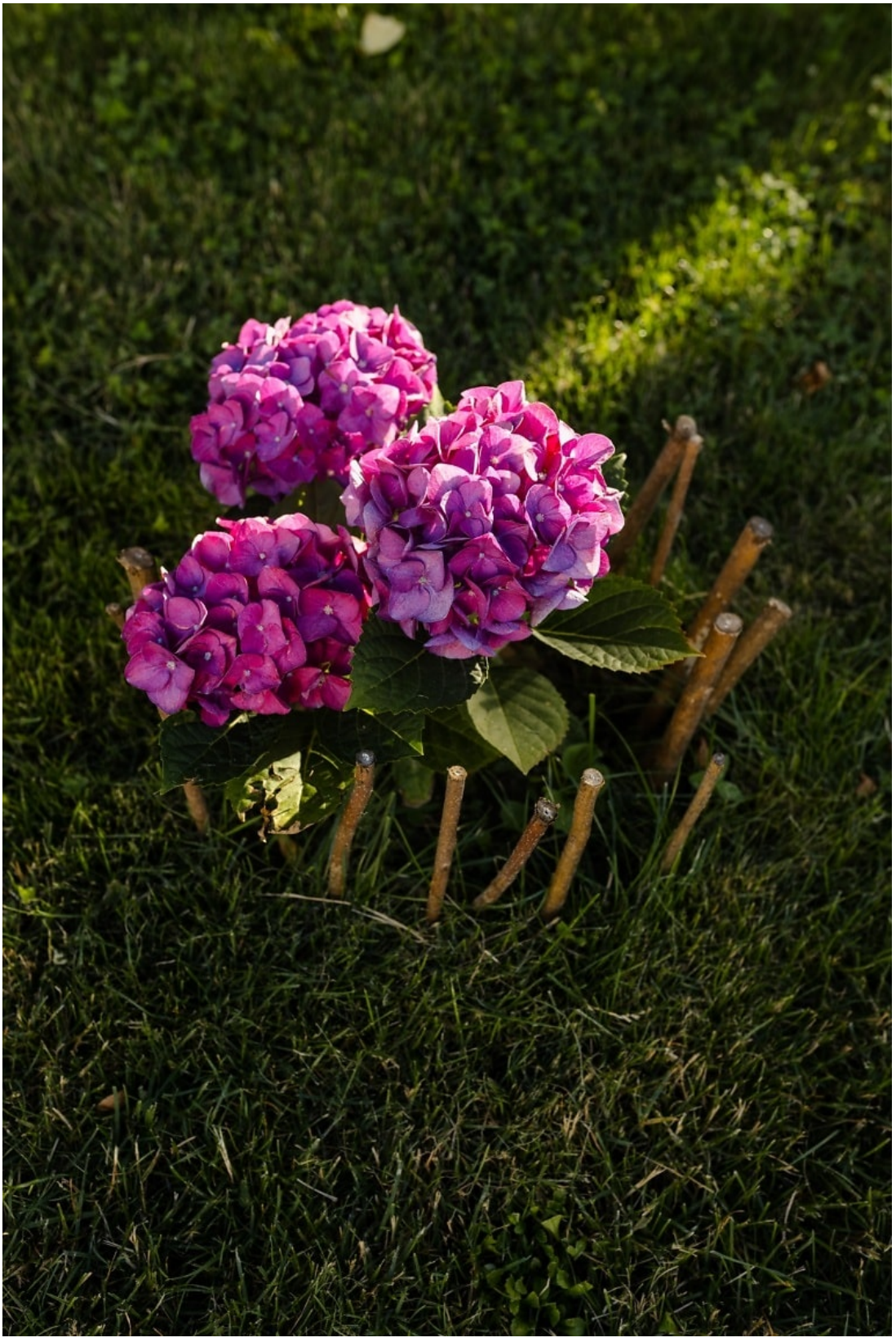

Figure 3

风格图与生成图（**已经燃尽了，效果不是很理想（哭）**）：

`set1`:

风格图:

Figure 4

生成图（有明显变化，但是为什么变蓝了？没办法，原图蓝色为主色调）：

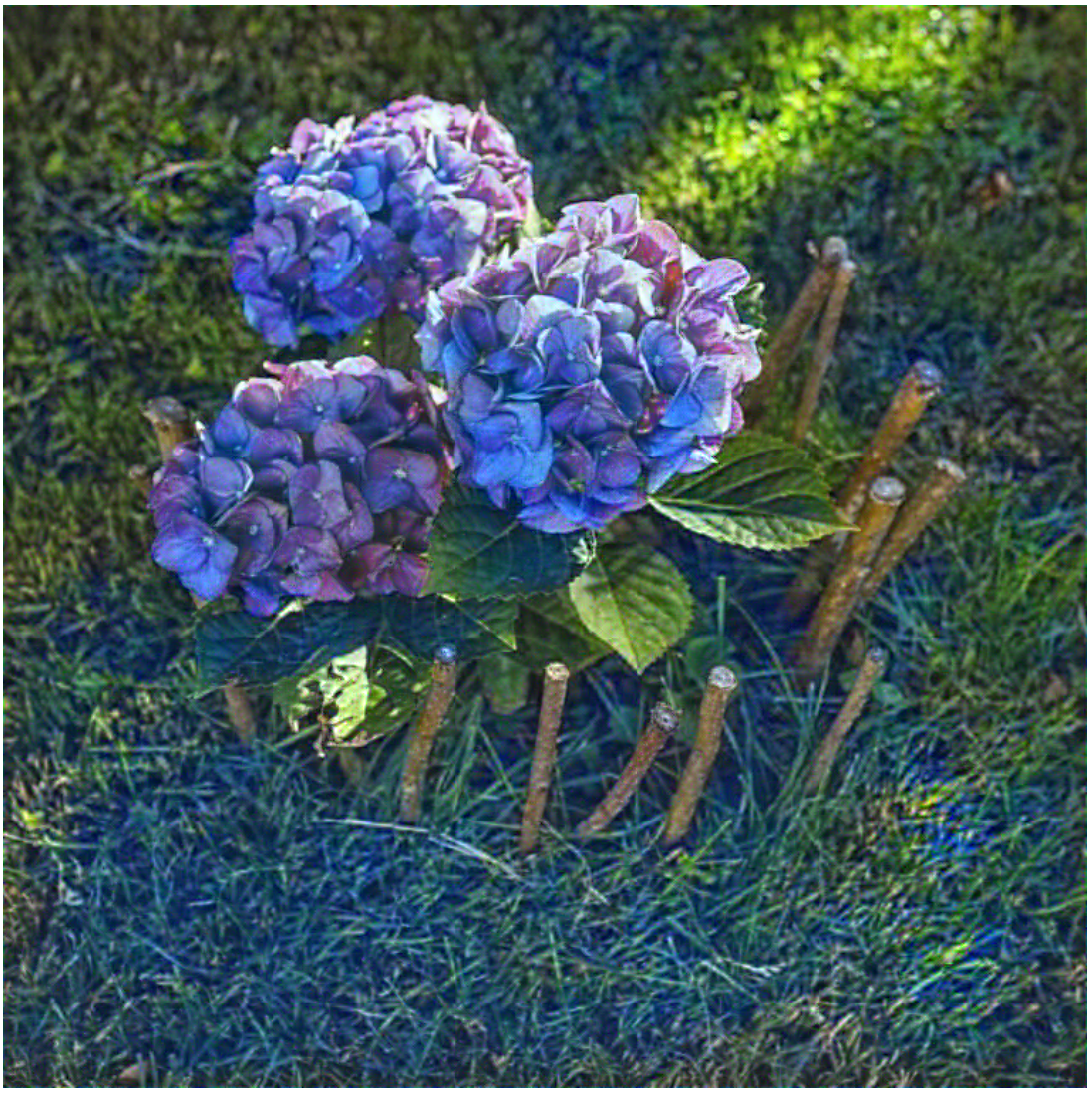

Figure 5

set2:

风格图:

Figure 6

生成图（效果来说不错的一张，缝合二次元柔和风格较好）：

Figure 7

风格图:

Figure 8

生成图（颜色终于不会大变了，夕阳感也很强，不过为什么感觉像是被氧化了？氧化的铜……（误 ）：

Figure 9

3.1 失败品

下面是失败品（战败CG？），可不看：

仅展示set1

内容损失权重和TV损失权重太小，风格损失权重太大的后果：

不是哥们，这给我干哪来了（

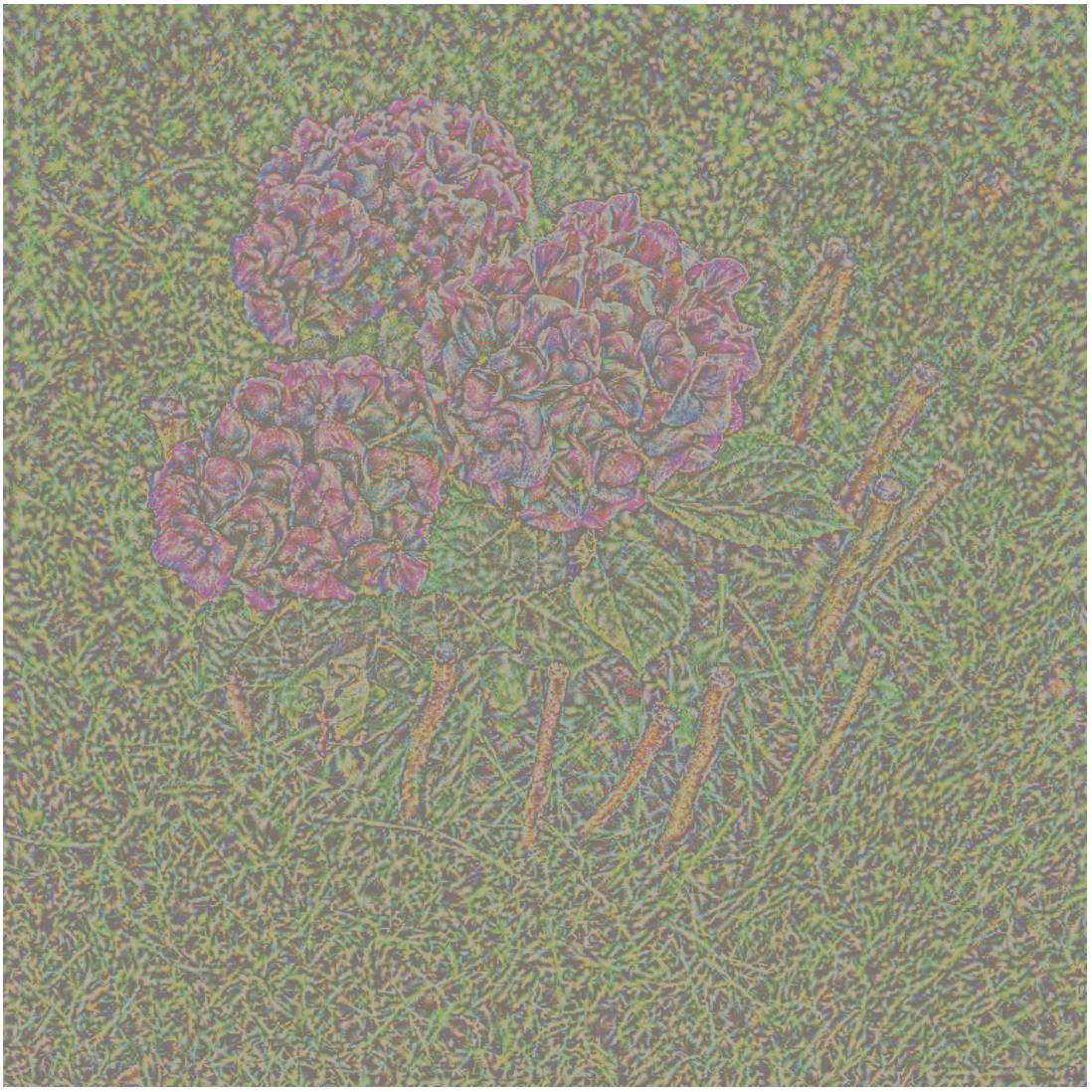

Figure 10

忘记clamp (0, 1) ：

张量数据溢出被截断，出现明显红色噪点：

Figure 11

3.2 回答问题

3.2.1 风格图和内容图相差太大会有影响吗?

会，而且是最容易被低估的因素

为什么:

1. **优化目标冲突**

   风格迁移在同时做两件事：让生成图的内容特征像内容图、Gram 矩阵像风格图

   如果：色调接近、，冲突程度小 —— 两个目标大致同向

   如果：一方极简、一方繁复，冲突程度**大** —— 优化器只能"二选一"

2. **颜色会被"拉走"**

   正如set1里面变蓝色一样，紫红花变蓝花这不是单单风格上的变化，而是已经涉及到了内容的变化，我们要的是带有梵高风格的花，而不是颜色彻底改变

   而set3里面变成夕阳风格，但是花还是可以看出是紫红色，内容被风格拉走的程度少

   原因：风格图的 Gram 矩阵编码了**颜色通道之间的相关性**。如果风格图是"蓝色为主"，那么 Gram 矩阵就是"蓝通道和蓝通道强相关"。优化器为了让 Gram 匹配，**会把生成图的颜色分布向风格图靠**

   所以内容图本身的颜色是不是"强色"很重要

3. **纹理密度差异**

   风格图纹理密度**过密**：容易出**高频噪点**

   风格图纹理密度**过疏**：风格压不住内容细节，**几乎看不出迁移**

3.2.2 三个损失的数学原理

上面有解释，防止没看到，再说一遍：

$$L_{total} = \alpha L_{content} + \beta L_{style} + \gamma L_{tv}$$

可以看到，这个公式由三部分组成：

内容损失（Content Loss），风格损失（Style Loss），总变分损失（Total Variation Loss）

总损失就是三者的加权和

内容损失（Content Loss）：

在本题中我们仅用relu4_2的结果来衡量内容

度量：生成图的"形状布局"和内容图差多少

$$L_{content} = \frac{1}{2} \sum_{i,j} (F_{ij} - P_{ij})^2$$

说人话：把生成图和内容图都送进 VGG，在 relu4_2 取特征，逐元素相减，平方求和

```
c = sum(F.mse_loss(feats[i], content_target[i]) for i in CONTENT_LAYERS)
```

Fence 2

风格损失（Style Loss）：

在本题中我们用relu1_1，relu2_1，relu3_1，relu4_1，relu5_1一起来衡量风格

度量：生成图的风格和风格图差多少

关键区别：它不比较特征图本身，而是比较 Gram 矩阵：

$$L_{style} = \sum_l w_l \cdot L^l_{style} \qquad L^l_{style} = \frac{1}{4N_l^2 M_l^2} \sum_{i,j} \left(G^l_{ij} - A^l_{ij}\right)^2$$

说人话：生成图和风格图经过relux_1后 Gram 矩阵差的元素平方和，再加到一起

```
s = sum(F.mse_loss(gram_matrix(feats[i]), gram_matrix(style_target[i]))
        for i in STYLE_LAYERS)
```

Fence 3

总变分损失（Total Variation Loss）：

$$L_{tv} = \sum_{i,j} \left(|x_{i,j+1} - x_{i,j}| + |x_{i+1,j} - x_{i,j}| \right)$$

说人话：遍历每个像素，算它和右边、下边邻居的差的绝对值，求和

它和其他两个损失的关键不同：

TV比较对象是**一张图内部**相邻像素，而不是**两张图之间**

```
t = (torch.mean(torch.abs((gen_normed[:,:,:-1,:] - gen_normed[:,:,1:,:]))) +
     torch.mean(torch.abs((gen_normed[:,:,:,:-1] - gen_normed[:,:,:,1:]))))
```

Fence 4

3.2.3 Gram 矩阵的作用？为什么内积能表示相似度？

Gram 矩阵的定义：

设某层特征图 reshape 成 F，形状 `(C, M)`：

- C = 通道数（如 512）
- M = 高 × 宽（如 96×96 = 9216

$$G = F \cdot F^T$$

$$L_{style} = \frac{1}{4N_l^2 M_l^2} \sum_{i,j} \left(G^l(i,j) - A^l(i,j)\right)^2$$

G输出形状：（c，c）

Gram 矩阵的作用：

把"逐位置的特征图"压缩成"通道之间的共现关系矩阵"，从而得到一种与位置无关的"风格指纹"

G输出形状：（c，c）与特征图大小M无关

为什么内积能表示相似度：

G 的第 (i,j) 个元素是

$$G_{ij} = \sum_k F_{ik} \cdot F_{jk}$$

即**第 i 个通道和第 j 个通道，在每一个空间位置上的值相乘，再全部相加**

内积是"逐位相乘再相加"，因此：

- 只有当两个通道**在同一个空间位置上同时有响应**时，这一项乘积才非零，才对总和有贡献
- 如果两个通道的活跃位置互不重叠，每一项乘积都接近 0，内积也就接近 0

换言之，**内积在"两个通道同时有响应的位置"上累加，响应位置越重合、响应强度越大，内积就越大。所以内积的大小直接反映了两个通道的共现程度**

3.2.4 风格损失与内容损失的差异，为什么gram矩阵更适合捕捉风格?

	内容损失	风格损失
比较什么	特征图**本身** (C,H,W)	**Gram 矩阵** (C,C)
关心位置吗	**关心**（逐元素比）	**完全不关心**
取自	1 层（relu4_2）	5 层
约束强度	**强**（要求布局一致）	**弱**（只要求统计一致）
作用	保住"画的是什么"	搬来"怎么画的"

Table 7

Gram 更适合捕捉风格：

1. 风格本质上是"统计性的"：

   "梵高的画风"不是"第某行第某列应该是什么颜色"，而是：笔触是粗还是细、颜色饱和度如何、哪些颜色倾向一起出现**这些都是统计特征，而 Gram 矩阵正好编码统计特征**

2. 与图片尺寸无关：

   Gram 形状 (C,C) 只取决于通道数，所以内容图和风格图尺寸不同也能直接比较

3. **我们本身需要的是内容提供骨架，风格提供皮肤**

   也就是**内容损失需要与空间信息强相关的量来强约束，防止图片骨架崩坏，内容丧失**

   而风格来提供皮肤，意味着其并不强依赖于空间信息，而是不同尺度的统计特征，这些正是Gram 的舒适区

3.2.5 为什么要把目标特征从计算图中分离?

因为内容图和风格图是"目标"，不是"待优化的变量"

只有在需要改变内容图和风格图参数时，才要计算图的数据来计算

但是**我们不需要**，这不是我们要改变的，要改变的是生成图的像素

不分离反而容易：

1. **可能改坏原图**（不小心误用）
2. **白算**（反向传播会为内容图和风格图的目标特征也算梯度，但我们根本不用）

3. 显存/内存占用大

3.2.6 设为评估模式和冻结参数是一回事吗?

不是，这是两件独立的事

	冻结参数	**评估模式**
代码	`p.requires_grad = False`	`model.eval()`
影响什么	**参数会不会被更新**	**层的行为方式**
具体作用	反向传播不算梯度、优化器不改它	Dropout 关闭、BatchNorm 用训练时累计的滑动平均

Table 8

3.2.7 如何从模型中抽取合适的层?依据是什么?

依据：感受野 + 空间分辨率

内容层为什么选 `relu4_2`：

候选	**问题**
太浅（relu1_1、relu2_1）	**没有语义**，提取到了纹理特征，会强迫生成图的每个边缘都和内容图一模一样
太深（relu5_1）	只有 48×48，**空间分辨率太低**，位置约束太弱，形状会飘
relu4_2	**96×96、512 通道 —— 既有语义（知道"是花"），又有足够分辨率（知道"在哪"）**

Table 9

风格层为什么用 5 层：

"风格"是多个尺度上的东西

层	**特征图**	**捕捉的风格元素**
relu1_1	768×768	**最细的笔触、色彩斑点**
relu2_1	384×384	小尺度纹理
relu3_1	192×192	中等纹理
relu4_1	96×96	大一些的结构
relu5_1	48×48	**整体色彩布局**

Table 10

只用浅层	**只用深层**	**5 层全用**
只有细碎纹理，缺大色块	只有粗糙色块，缺笔触细节	**笔触和色块都有**

Table 11

3.2.8 L-BFGS 相比 Adam、SGD 的特点

	SGD	Adam	L-BFGS
阶数	一阶	一阶（+ 自适应步长）	**二阶（拟牛顿）**
用到的信息	梯度	梯度 + 平方的滑动平均	梯度 + **最近几步的曲率估计**
步长怎么定	固定lr（需要调）	自适应（每坐标不同）	**线搜索自动找**
每"步"的计算量	1 次前向+反向	1 次	**5~20 次**（线搜索）
收敛所需步数	数百 ~ 数千	数百	**十几 ~ 几十**
适合场景	**大规模数据集**	**大数据、小批量、有随机性**	**小模型、预训练微调、要求精度、全批量**

Table 12

为什么风格迁移适合 LBFGS（单张图片、全批量、像素级优化）：

1. **优化变量极少，二阶方法负担得起**
2. **每次迭代都是全批量，没有随机梯度噪声**
3. **收敛所需的迭代次数很少**

   L-BFGS 利用曲率信息，下降速度快

3.2.9 风格迁移主循环的流程

1. 初始化生成图为内容图
2. 得到生成图和风格图经过vgg后各层的特征张量
3. 在闭包函数closure内，执行下面的逻辑：

   清空梯度记录

   把生成图截断为[0,1]

   得到归一化的生成图经过vgg后各层的特征张量

   计算**内容损失，风格损失，总变分损失**和**总损失**

   返回总损失
4. 每一步优化器**L-BFGS**调用闭包函数closure，得到总损失，然后反向传播计算梯度，调整生成图像素
5. 使用早停，当连续*patience*=5轮后没进步终止训练流程
6. 最终返回生成图的张量，通过to_PIL_image保存图像到指定路径

3.2.10 前向传播和反向传播的流程

前向：

gen 形状 (1, 3, 768, 768)

1. to_norm() → (1, 3, 768, 768) 归一化
2. VGG19.features 逐层前向
   1. 到 relu1_1 (索引 1) → 存 (1, 64, 768, 768) ← 风格
   2. 到 relu2_1 (索引 6) → 存 (1, 128, 384, 384) ← 风格
   3. 到 relu3_1 (索引 11) → 存 (1, 256, 192, 192) ← 风格
   4. 到 relu4_1 (索引 20) → 存 (1, 512, 96, 96) ← 风格
   5. 到 relu4_2 (索引 22) → 存 (1, 512, 96, 96) ← 内容
   6. 到 relu5_1 (索引 29) → 存 (1, 512, 48, 48) ← 风格
   7. 到索引 29 就 break (后面的层不用跑，省算力)
3. 计算三个损失和总损失

反向：

`loss.backward()`

1. 沿计算图回传：总损失，三个损失
2. 逐层计算梯度，直到 `x`
3. 更新 `x`

3.2.11 有哪些超参数？各个损失的权重该如何设置？

超参数清单

类别	参数	本题取值	作用
尺寸	`IMG_SIZE`	**768**	生成图边长
迭代	`NUM_ITERS`	**100**	LBFGS 外层步数
快照	`NUM_SNAPSHOTS`	**8**	过程图张数（对数间隔）
LBFGS	`LBFGS_LR`	**1.0**	线搜索的初始尺度
	`LBFGS_MAX_ITER`	**20**	每次 step 内部最多几次子迭代
	`LBFGS_HISTORY`	**10**	用最近多少步梯度近似曲率
权重	`CONTENT_WEIGHT` (α)	**1.0**	内容强度
	`STYLE_WEIGHT` (β)	**1e4**	风格强度
	`TV_WEIGHT` (γ)	**1e-4**	平滑程度
层	`CONTENT_LAYERS`	`[22]`	内容层 (relu4_2)
	`STYLE_LAYERS`	`[1, 6, 11, 20, 29]`	风格层 (relu1_1~5_1)
早停	`patience`	**5**	连续几步没改善就停
	`min_delta`	**1e-4**	"算有改善"的阈值

Table 13

各个损失的权重怎么选：

损失的量级取决于归约方式和 Gram 归一化，也就是看公式怎么填的：

写法	内容损失量级
`((F-P)**2).sum()`	10^2
`F.mse_loss(F, P)`	10^{-2}

Table 14

正确的做法：看加权后的占比

本题数据（$\beta = 10^4$）：

set	加权内容	加权风格	风格占比
set1	0.0386	0.0294	**76.0%**
set2	0.0992	0.4553	**82.1%**
set3	0.0460	0.0317	**77.5%**

Table 15

风格占 76~82%，所以画面风格化很强

β 不同取值的影响：

β	**风格占比**	**视觉表现**
1e4	~82%	风格很强，但容易出现高频噪点
3e3	~60%	中等
1e3	~35%	风格较弱
3e2	~15%	几乎看不出迁移

Table 16

γ 不同取值的影响：

γ	**孤立像素占比（越低越平滑）**
0（不加 TV）	23.7%
1e-4	22.7%
1e-3	22.1%
1e-2	21.6%
1e-1	**11.8%**

Table 17

γ 越大越平滑，但太大会糊掉细节

3.2.12 总损失越小迁移效果越好吗？如何调参？

不一定，损失函数只衡量"VGG 特征层面的匹配程度"，看不见：

1. 像素是否在合法范围内（需要及时补clamp(0, 1)）
2. 风格强度是否合适
3. 人眼觉得好不好看（过曝、糊、比例失调）

而且，总损失是衡量生成图与内容图和风格图的差异

总损失过小，可能意味着与内容图差异极小，风格反而学不到

真正效果还需人眼来评判，而不是将总损失小作为迁移效果好的最终证明

如何调参：

1. 选一个参数为基准（α = 1.0）
2. β、γ 先给个数量级（1e4、1e-4）
3. 扫 β（风格强度）

   1e4 → 3e3 → 1e3 → 3e2

   每个值看：风格是否明显？内容是否还认得出？
4. 扫 γ（平滑程度）

   0 → 1e-4 → 1e-3 → 1e-2 → 1e-1

   每个值看：噪点少了？细节糊了没？变化大不大？
5. 选取眼睛认为最可以的一组参数（也可以叫上好同学）

4. 选做题

4.1 风格强度控制

4.1.1 参数设计

风格强度的"旋钮"本质就是风格损失权重 β，但直接暴露 β 不好用 —— 它的取值是 $10^2 \sim 10^5$ 这种量级，调起来没有"手感"

所以定义一个 $0 \sim 1$ 的直观参数 s（`STYLE_STRENGTH`），按**对数间隔**映射到 β：

$$\beta = 10^{2+2s}$$

s	0.00	0.25	0.50	0.75	1.00
β	10^2	3.16×10^2	10^3	3.16×10^3	10^4

Table 18

4.1.2 实验设置

项	设置
固定	$\alpha = 1.0$、$\gamma = 10^{-4}$、尺寸 768、100 步、LBFGS
变化	β（由 s 决定），取 $s = 0.25/0.50/0.75/1.00$
数据	set1（风格=星空）、set2（风格=二次元），共 8 次

Table 19

4.1.3 结果

set1（风格图 = 梵高《星空》）

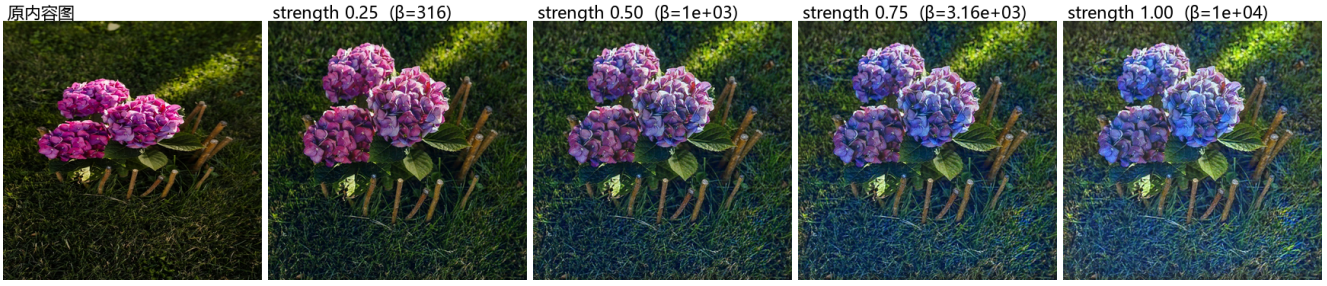

Figure 12

set2（风格图 = 二次元插画）

Figure 13

损失数据：

set	s	β	内容损失	风格损失	加权风格占比	总损失
set1	0.25	316.2	**0.001146**	3.736e-05	90.9%	0.0130
set1	0.50	1000	**0.004019**	2.609e-05	86.5%	0.0302
set1	0.75	3162	**0.012671**	1.816e-05	81.9%	0.0701
set1	**1.00**	1e4	**0.038687**	1.226e-05	76.0%	0.1613
set2	0.25	316.2	**0.003204**	9.739e-05	90.5%	0.0340
set2	0.50	1000	**0.010256**	7.491e-05	87.9%	0.0852
set2	0.75	3162	**0.030624**	5.989e-05	86.1%	0.2200
set2	**1.00**	1e4	**0.098317**	4.500e-05	82.1%	0.5484

Table 20

4.1.4 分析：为什么会这样

两条单调趋势：

	$s = 0.25 \to 1.00$
内容损失	set1 涨 **33.8 倍**（0.0011 → 0.0387）；set2 涨 **30.7 倍**（0.0032 → 0.0983）
风格损失	set1 降 **3.05 倍**（3.74e-05 → 1.23e-05）；set2 降 **2.16 倍**（9.74e-05 → 4.50e-05）

Table 21

原因是两个损失在争夺同一块像素：

总损失的梯度是

$$\nabla L_{total} = \alpha \nabla L_{content} + \beta \nabla L_{style} + \gamma \nabla L_{tv}$$

β **决定了"风格项在梯度里的话语权"**：

β	梯度里谁主导	优化器的行为	后果
小（s=0.25）	**内容项主导**	只敢小幅修改像素	内容几乎不变（0.0011），但 Gram 没被拉过去（风格损失高）
大（s=1.00）	**风格项主导**	不惜大幅改动像素来凑 Gram	Gram 贴得很紧（风格损失低），但内容被破坏（0.0387）

Table 22

像素总量固定，用来满足风格的部分多了，留给保持内容的部分就少了

视觉上完全对应：

s	画面
0.25	基本还是原图的粉色绣球，只有极轻微的色调偏移和纹理
0.50	开始出现蓝紫色调，草地有笔触感
0.75	明显偏蓝紫，笔触清晰，但花瓣结构还能看清
1.00	整体是星空的蓝紫色调，笔触强烈，原图的粉色基本消失

Table 23

华生你发现了盲点："加权风格占比"这一列随 β 增大反而**下降**（90.9% → 76.0%），看起来反直觉

原因是**分母也在变**：β 增大时风格损失本身被压低了（3.74e-05 → 1.23e-05），所以"占比"不是衡量风格强度的好指标

要判断风格强度，应该直接看 β **和画面对比，而不是看占比**

4.2 2. 多风格融合

4.2.1 思路：把风格损失拆成两项

单风格时，风格损失只有一项：

$$L_{style} = \beta \cdot L_{style}^{(1)}$$

两种风格时，拆成两项，各配一个权重：

$$L_{style} = \beta_1 \cdot L_{style}^{(1)} + \beta_2 \cdot L_{style}^{(2)}$$

但直接把 β_1、β_2 都当自由参数会有个问题：总风格力度不清楚 —— $\beta_1 = \beta_2 = 5000$ 和 $\beta_1 = \beta_2 = 10000$ 哪个"更风格化"？

所以固定总力度，只用一个占比 w 来分配：

$$\beta_1 = \beta_{total} \cdot w, \qquad \beta_2 = \beta_{total} \cdot (1 - w)$$

w	β_1（二次元）	β_2（夕阳）	偏向
0.25	2500	7500	夕阳
0.50	5000	5000	五五开
0.75	7500	2500	二次元

Table 24

取 $\beta_{total} = 10^4$ —— 与基础版单风格时一致

4.2.2 实验设置

项	设置
固定	$\alpha = 1.0$、$\beta_{total} = 10^4$、$\gamma = 10^{-4}$、尺寸 768、100 步、LBFGS
变化	$w = 0.25/0.50/0.75$
输出	每组一张 `result.png` + `loss.csv`

Table 25

4.2.3 结果

Figure 14

(从左到右：原内容图 / 纯二次元 / $w = 0.25$ / $w = 0.50$ / $w = 0.75$ / 纯夕阳)

损失数据：

w	β_1	β_2	内容损失	风格1损失	风格2损失	总损失	步数
0.25	2500	7500	0.037330	**1.2697e-04**	**2.3154e-05**	0.5284	100
0.50	5000	5000	0.044609	**8.6061e-05**	**4.5663e-05**	0.7033	100
0.75	7500	2500	0.064879	**5.8273e-05**	**8.3878e-05**	0.7117	82

Table 26

4.2.4 分析：实验现象

现象一：两个风格损失此消彼长

w **增大（二次元权重变大）**	**结果**
风格1损失（二次元）	**单调下降** 1.27e-04 → 8.61e-05 → 5.83e-05（降 2.18 倍）
风格2损失（夕阳）	**单调上升** 2.32e-05 → 4.57e-05 → 8.39e-05（涨 3.62 倍）

Table 27

两个权重各自独立生效：谁权重大，生成图就更像谁

```
w=0.25:  风格1 1.27e-04  >  风格2 2.32e-05    ← 更像夕阳
w=0.50:  风格1 8.61e-05  >  风格2 4.57e-05    ← 还是偏向夕阳
w=0.75:  风格1 5.83e-05  <  风格2 8.39e-05    ← 更像二次元
```

Fence 5

交叉点落在 $w \approx 0.6 \sim 0.7$ **之间** —— 也就是说，**要让两种风格被"同等程度地满足"，**w **并不是 0.5，而要略大一些**

现象二：内容损失随 w **增大而上升**

w	**内容损失**
0.25	0.037330
0.50	0.044609
0.75	0.064879

Table 28

两种风格的"冲突程度"不同：夕阳的色调和原图（暖光草地、粉紫花）更接近，二次元偏冷偏蓝、和原图差异更大

加重二次元权重时，要偏离原图更多才能匹配上它的 Gram，内容损失就更高

现象三：融合结果确实"介于两者之间"

对比最左和最右两张单风格结果：

	画面特征
纯二次元	明显偏冷蓝，花瓣变成蓝紫色，笔触碎
$w = 0.25$	**暖橙色调（夕阳的成分），但花瓣上带一点冷的紫蓝（二次元的成分）**
$w = 0.50$	暖冷平衡
$w = 0.75$	偏冷蓝，二次元风格强
纯夕阳	强烈的暖橙/褐色，整体压暗

Table 29

说明融合确实发生了

4.3 3. 局部风格迁移（难）

4.3.1 手搓轮廓

Figure 15

mask 的覆盖率：

区域	占全图
花朵（mask ≥ 0.9）	**14.72%**
羽化过渡带（0 < mask < 0.9）	4.57%
纯背景（mask = 0）	**80.71%**

Table 30

4.3.2 把 mask 接进损失函数

一：内容损失只在 mask 内算

$$L_c = \frac{\sum_{i,j} M_{ij} \, (F_{ij} - P_{ij})^2}{\sum_{i,j} M_{ij}}$$

（除以 mask 内的元素数，这样损失量级和"整张图算"一致，α, β 不用重新调）

二：风格损失也只在 mask 内算 —— 先把特征图乘上 mask，再算 Gram

$$F^{(m)} = F \odot M, \qquad G = \frac{F^{(m)} \cdot F^{(m)\mathsf{T}}}{C \cdot |M|}$$

三：输出时硬合成

$$\text{result} = \text{gen} \odot M + \text{原图} \odot (1 - M)$$

4.3.3 结果

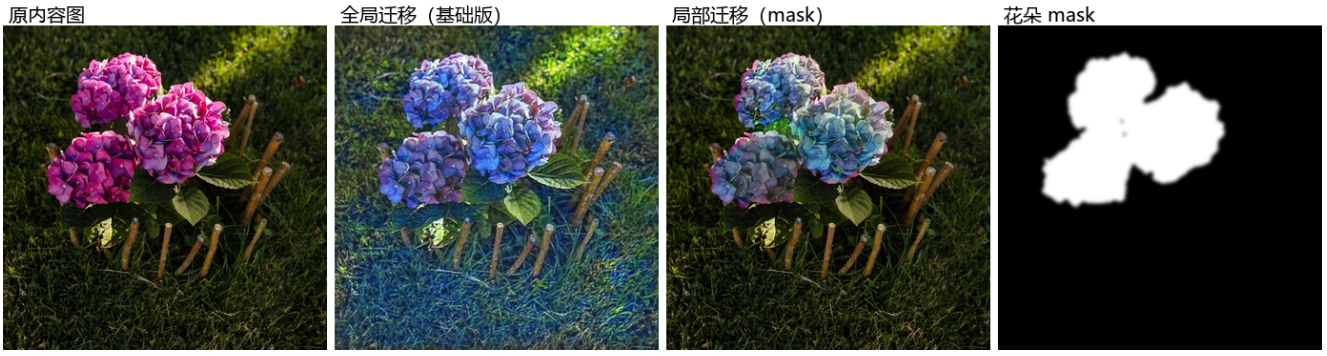

Figure 16

(从左到右：原内容图 / 全局迁移 / 局部迁移 / 花朵 mask)

对比非常直观：

	花朵	**草地**
全局迁移	变成星空蓝紫	**也变成蓝绿色**
局部迁移	变成星空蓝紫	**保持原本的黄绿色**

Table 31

损失数据：

内容损失	**风格损失**	**TV 损失**	**总损失**	**步数**	**用时**
0.1823	3.498e-05	0.32848	0.5322	94	190 秒

Table 32

代码仓库地址，可以看看~~：

https://github.com/neri-mini/style_transfer